Extracting Functional Modules from Flattened Gate-Level Netlist

Yiqiong Shi, Bah-Hwee Gwee, Ye Ren, Thet Khaing Phone and Chan Wai Ting
Integrated Systems Research Laboratory
School of Electrical and Electronic Engineering
Nanyang Technological University, Singapore

Abstract—**A generic and highly versatile method for extracting functional modules from a flattened gate-level netlist is proposed. The proposed method requires no prior knowledge about the netlist under analysis and is applicable to circuits targeting diverse applications. It is fully automated and employs a highly compact module library containing only single generic model for each common function type with arbitrary data width. Experiment results depict the efficacy of the proposed method and its embodied techniques.**

Keywords-Verification; semantic equivalence checking; subcircuit recognition

I. INTRODUCTION

Highly complex multimillion-gate state-of-the-art ICs are challenging to computer-aided design (CAD) tools and inevitably call for the 'divide and conquer' strategy. As part of the design flow, specifically for formal verification [1] and for high-level design-for-testability (DFT) [2], it is imperative that modern CAD tools are able to comprehensively extract high-level functional modules (of a circuit) from its low level description. In other IC design related processes/issues, functional module extraction is also employed in Trojan detection [3], copyright infringement investigation and competitive analysis [4].

Functional module extraction methods in the literature can be largely divided into two categories, namely semantic equivalence checking [5] and syntactic equivalence checking [6, 7]. Both techniques attempt to recognize and locate subcircuits in the input netlist that are equivalent to known modules in a module library. Each technique has its merits and disadvantages. In general, the semantic equivalence checking technique identifies the functional equivalence between two subcircuits and is hence implementation independent. However, as these subcircuits have to be represented in their canonical form, whose complexity grows exponentially with the number of inputs, semantic equivalence checking is practically applicable only to circuits with small number of inputs and small size [8].

On the other hand, syntactic equivalence checking technique in general recognizes structural equivalence between two subcircuits. As this technique is essentially graph-based, it is less restricted by the size of the circuits but is highly implementation dependent. In order for the syntactic equivalence checking technique to identify subcircuits without knowledge pertaining to the implementation features of the input netlist, the associated module library must contain all possible implementations of all function – a highly unrealistic scenario. In short, neither of the two aforesaid techniques is in general powerful and versatile enough to handle circuits with large size as well as unknown implementations. In view of these limitations, researchers have proposed methodologies that attempt to mitigate the limitations of the individual techniques. Chisholm et al. [9] reported a methodology that employs both techniques to derive a module-level description from a gate-level netlist. This methodology, nonetheless, requires prior knowledge about the circuit being analyzed.

In addition to the intrinsic limitations of the two techniques, functional module extraction is further complicated if the data width of the modules is unknown. In this case, the associated module library would have to contain models for all possible data width of a given function – this is impractical for reasons of the size of the associated module library and the ensuing search time to identify the specific functional modules in question.

Hansen et al. [10] suggested the use of a series of recognition steps to identify high-level structures in logic circuits. Although this suggested technique does show promise, for example in the ISCAS-85 benchmarks, the process involved is highly intuitive (hence requiring substantial experience and know-how) and the amount of manual work may be intolerable in complex designs.

This paper describes a novel method that extracts functional modules from a flattened gate-level netlist, that does not suffer from the limitations of the individual aforesaid techniques and that is more versatile than reported techniques. Specifically, it is not restricted by the size, implementation style or targeting application of the circuits and it requires no prior knowledge about the input netlist whatsoever. Further, it is fully automated - although user input can facilitate to speed up the process. The generality of the proposed method is, in part, achieved by means of employing a highly compact module library that comprises only single generic model for each type of function with arbitrary data width. In addition, the proposed method employs several novel techniques to speed up the basic exhaustive-search based candidate subcircuit (CS) enumeration algorithm, and to reduce the number of candidate subcircuits to be compared with the library modules.

978-1-4673-1157-1/12/$31.00 © 2012 IEEE

This paper is organized as follows. Section II describes the overall workflow of the proposed functional module extraction method. Section III details the Subcircuit Recognition (SR) algorithm therein, which is the most computational intensive stage. The implementation and results are presented in Section IV, followed by the conclusions and future work in Section V.

II. FUNCTIONAL MODULE EXTRACTION METHOD

The proposed method for extracting functional module from flattened gate-level netlist is shown in Fig. 1. The various tasks and their respective products will now be delineated in detail.

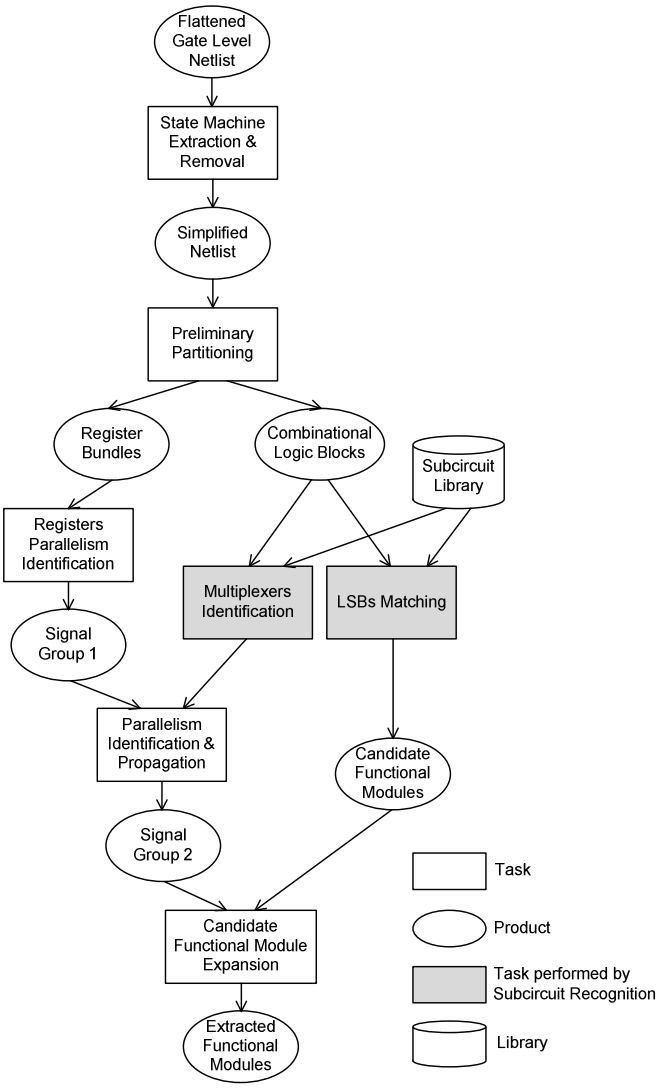


Figure 1. Overview of proposed functional module extraction method.

A. State Machine Extraction & Removal

The method of extracting state machine from flattened gate-level netlist has been reported [11]. The state machine extraction method involves mainly two major steps, the identification of the true state registers and the grouping of the state registers (and their respective combinational logic) into state machines. By extracting and removing the state machines in the beginning of the analysis, the flattened gate-level netlist is simplified whereby only the datapath which contains the functional modules of interest remains.

B. Preliminary Partitioning

The purpose of the preliminary partitioning task is to extract combinational logic blocks from the simplified netlist. The simplified netlist is first converted to di-graph matrix such that graph theory and associated algorithms can be applied. All connections into and out from the registers therein are then disconnected. When the connected components algorithm is applied to the modified di-graph matrix, the ensuing components are essentially connected combinational logic blocks. Registers driving and driven by the same combinational logic blocks are then collected into register bundles.

C. Least Signaficant Bits (LSBs) Matching

The Least Significant Bits (LSBs) matching task is one of the key tasks in the proposed method. It is carried out by the Subcircuit Recognition algorithm (see Section III below). The two major challenges tackled by the proposed LSBs matching task are the unknown data width of the functional modules and numerous possible implementations of each function.

The first challenge is tackled by representing the functional modules with only a few LSBs (as opposed to the entire data width). Hence only one generic model of one type of function (of arbitrary data width) needs to be stored in the module library. The few LSBs can be used to model the functional modules for two reasons. First, the intermediate signals in most of the calculations/operations propagate upwards from the least significant bits to the most significant bits (MSBs). In other words, the value as well as the number of the higher order bits will generally not affect the logic of the LSBs. Second, the few LSBs are generally sufficient to differentiate the different types of functions, e.g. an adder from a multiplier.

The second challenge is tackled by employing implementation independent semantic equivalence checking method in the SR algorithm (see Section III.A below). As discussed in Section I earlier, the semantic equivalence checking can only be used for circuits with a small number of inputs. Representing functional modules with only a few LSBs reduces the size and the number of inputs of the library modules to be recognized in the SR algorithm, and hence enables the effective use of semantic equivalence checking.

The product of this task is identified candidate functional modules (CFMs), e.g. adders and multipliers, with their LSBs appended. The CFMs will be expanded to their full data width later in candidate functional module expansion task.

D. Multiplexers Identification

The task of multiplexers identification also employs the semantic-equivalence-checking based SR algorithm as multiplexers can also be implemented in a number of different ways. Identifying multiplexers is crucial as they can be used in target circuit partitioning (indicating the boundary between one functional module and another) and parallelism identification

and propagation (indicating the correlation among different signals).

E. Parallelism Identification & Propagation

Signals are said to be parallel if they belong to the same data, i.e. they are different bits of a given multi-bit data. Registers, on the other hand, are said to be parallel if their input/output signals are parallel. Registers parallelism can be identified by grouping registers in the register bundles (produced by the preliminary partitioning task) according to their enable signals. This is because registers controlled by the same enable signal are generally more closely related. Similarly, multiplexers controlled by the same select signal are in general more closely related.

The parallelism identified from registers' enable signals and/or multiplexers' select signal can also be propagated upstream or downstream. For instance, if several multiplexers have their output signals identified to be parallel from the previous registers parallelism identification task, this parallelism can now be propagated upstream to their corresponding input signals.

The product of the registers parallelism identification task and the parallelism identification & propagation task is collection of parallel signals, called signal groups.

F. Candidate Functional Module Expansion

The next task is to expand the candidate functional modules to its full data width. Its inputs are the identified LSBs of CFMs from the LSBs matching task and the identified signal groups from the parallelism identification & propagation task. This task is performed by locating the identified LSBs of CFMs in the signal groups and identifying the bits parallel to the LSBs, which would most likely be their corresponding higher order bits. Consequently, it expands the CFMs to their full data width and represents them with their complete input/output signals.

A logic depth search is then performed on the target netlist from the LSB till all output signals are traced. The order of the output signals is determined by their corresponding logic depth from the LSB, with deeper logic depth (longer path) indicating higher order (closer to the MSB). The order of the input signals is then determined through backward tracking and is based on the fact that output bit *n* is normally only affected by input bits 0 to *n* (but not input bits of higher order).

G. Summary

In summary, the proposed method takes a flattened gate-level netlist and identifies the functional modules in the netlist, e.g. adders and multipliers. It is highly advantageous over reported algorithm as it requires no prior knowledge (e.g. type of functional module, native data width, implementation features) about the input netlist, resulting in high flexibility and versatility. Furthermore, it employs a highly compact module library containing only the logic of few LSBs of common functional modules and logic of multiplexers of different sizes. The runtime of the Subcircuit Recognition algorithm (see Section III later) is significantly reduced due to the small number and small size of the library modules. The runtime of the SR algorithm is further improved by reducing the size of the target circuit through a series of analysis and simplification steps.

III. Subcircuit Recognition Algorithm

The Subcircuit Recognition algorithm adopted in the proposed functional module extraction method, specifically in the LSBs matching task and multiplexers identification task, is shown in Fig. 2. The important features therein will now be delineated in detail.

1. Select library module to be matched;
2. Enumerate single-gate CSs from target (see Fig. 3(a));
3. Generate BDDs of single-gate CSs;
4. Compare BDDs of single-gate CSs with BDD of library module;
5. Remove matched single-gate CSs from target;
6. Enumerate multi-gate CSs from new target (see Fig. 3(b);
7. Cross match multi-gate CSs;
8. Perform Signature Testing on multi-gate CSs;
9. Generate BDDs of multi-gate CSs passing Signature Testing;
10. Compare BDDs of multi-gate CSs with BDD of library module;
11. Represent matched CSs with library module in the original target;

Figure 2. Proposed Subcircuit Recognition algorithm. (Note: CS – candidate subcircuit, BDD – Binary Decision Diagram)

A. Semantic Equivalence Checking

As discussed in Section I above, the proposed functional module extraction method assumes no prior knowledge about the implementation features of the input gate-level netlist. Syntactic matching techniques are therefore inappropriate as they would unrealistically require the library to contain all possible implementations of all the modules. The implementation independent semantic equivalence checking method is adopted and the Reduced Ordered Binary Decision Diagram (BDD) [12] is used to represent the library modules and candidate subcircuits in their canonical form. BuDDy [13] is used for BDD generation and manipulation.

B. Candidate Subcircuit Enumeration

After choosing the library module to be matched, e.g. a 2-to-1 multiplexer (with 3 inputs and 1 output), all candidate subcircuits with the same number of input(s) and output(s) in the target netlist will be enumerated using an exhaustive search. The CS enumeration algorithm is outlined in Fig. 3. As CS enumeration is the most time consuming stage of the proposed method, a few measures are proposed to reduce the runtime.

First, single-gate CSs are enumerated and matched first – this is shown in lines 2-4 in Fig. 2. All matched single-gate CSs are removed from the target netlist before multi-gate CSs are enumerated (see line 5 in Fig. 2) to reduce the size of the target netlist for shorter runtime (see Table 2 in Section IV later). This is because it is highly unlikely that a single gate/cell that already matches the library module will be combined with other gates/cells to build the same library module. Second, the multi-gate CS enumeration process is performed gate by gate and every gate will be removed from the target netlist once it is expanded and checked. This therefore avoids any duplication/repetition for better efficiency. Furthermore, removing gates one by one gradually reduces the size of the

target netlist, which also contributes to a shorter runtime. Third, an upper limit is set on the number of inputs of the candidate subcircuits. For example, any subcircuits with more than twice the number of inputs of the library module will not be further expanded. The upper limit is carefully chosen to avoid missing enumeration. The effect of the upper limit on runtime is also analyzed in Section IV later.

```
Let n = no. of gates/cells in the target;
for i = 1 to n, do {
        if input/output no. of gate i = input/output no. of library module,
                Add gate i to single-gate CS list;
}
Return single-gate CS list;
```

(a)

```
Let n = no. of gates/cells in the target;
for i = 1 to n, do {
        Let gate_neighbor_list = all vertices directed connected to gate i;
        Let m = no. of elements in the gate_neighbor_list;
        for j = 1 to m, do {
                Group neighbor j with the gate i to form new vertex;
                if input no. of new vertex <= 2 x input no. of library module,
                        Add new vertex to the expand_list;
        }
        while expand_list is not empty, do {
                Choose an element in the expand_list;
                if input/output no. of chosen element = input/output no. of library module,
                        Add chosen element to multi-gate CS list;
                Let neighbor_list = all vertices directed connected to chosen element;
                Let p = no. of elements in the neighbor_list;
                for k = 1 to p, do {
                        Group neighbor k with chosen element to form new vertex;
                        if input no. of new vertex <= 2 x input no. of library module,
                                Add new vertex to the expand_list;
                }
                Delete chosen element from the expand_list;
        }
        Delete gate i from the target netlist;
}
Return multi-gate CS list;
```

(b)

Figure 3. Proposed candidate subcircuit enumeration algorithm, (a) for single-gate candidate subcircuits, and (b) for multi-gate candidate subcircuits.

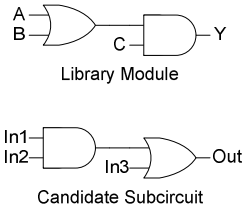
C. Candidate Subcircuit Cross Matching

Once the multi-gate candidate subcircuits with the same number of input(s) and output(s) as the library module are enumerated, they will be cross matched with each other (see line 7 in Fig. 2). The purpose of cross matching is to reduce the number of CSs that needs to be compared to the library module (see Section IV later), and it is syntactic equivalence based. This cross matching is performed by first sorting the enumerated CSs according to their number of gates/cells and sorting the gates/cells within one CS according to their types, followed by syntactic equivalence checking of CSs with the same number of gates/cells. If two CSs have the same number and type of gates/cells with the same interconnection topology, they are syntacticly equivalent and only one of the two needs to undergo the remaining steps of the Subcircuit Recognition algorithm. Note that no library is required in the syntactic equivalence checking for cross matching the CSs.

D. Signature Testing

The purpose of performing Signature Testing on CSs (see line 8 in Fig. 2) is not only to reduce the number of CSs requiring BDD comparison (by removing CSs that fails at least one of the tests) but also to reduce the number of input permutations of CSs requiring BDD comparison (by short listing input permutations that pass the tests). The Signature Testing algorithm adopted in the Subcircuit Recognition algorithm is based on the idea described by Eckmann and Chisholm [8]. An additional checking step is however proposed to improve the efficiency of the original algorithm as illustrated in Fig. 4.

Fig 4(a) shows an example library module with 3 inputs and 1 output, and a corresponding example candidate subcircuit. The results of Signature Testing (the signatures) of the two circuits are shown in Fig. 4(b) and (c) respectively. It can be seen that the CS passes the 'All 1' and 'All 0' tests, and the suspect sets of the three inputs of the library module (A, B and C) after the 'Single 1' tests are given in Fig. 4(d). A closer look at Fig. 4(d) reveals that the CS can never match the library module because there are three inputs (A, B and C) but only two suspects (In1 and In2), and there is no actual need to perform the 'Single 0' tests. This example depicts that the mere existence of the suspect sets does not always indicate a CS passing the Signature Testing. The proposed extra checking step involves requiring the inspection of the suspect sets after the 'Single 1' tests to determine the necessity for the 'Single 0' tests and also after the whole Signature Testing to determine if the CS passes the Signature Testing with realistic suspect sets. Consequently, the Signature Testing is more effective in reducing the number of CSs requiring BDD comparison in the subsequent steps.



(a)

Test	Result
All 1	1
All 0	0
Single 1 – A	0
Single 1 – B	0
Single 1 – C	0
Single 0 – A	1
Single 0 – B	1
Single 0 – C	0

(b)

Test	Result
All 1	1
All 0	0
Single 1 – In1	0
Single 1 – In2	0
Single 1 – In3	1
Single 0 – In1	1
Single 0 – In2	1
Single 0 – In3	1

(c)

After 'Single 1' tests –
Suspect Set (A) = {In1, In2}
Suspect Set (B) = {In1, In2}
Suspect Set (C) = {In1, In2}

(d)

Figure 4. Example illustrating proposed Signature Testing algorithm, (a) an example library module and a corresponding example candidate subcircuit, (b) signatures of library module in (a), (c) signatures of candidate subcircuit in (a), and (d) suspect sets after 'Single 1' tests.

IV. Implementation and Results

The proposed method for functional module extraction has been implemented and verified with a few benchmark circuits. The candidate subcircuit enumeration algorithm and Signature Testing algorithm are implemented in GNU Octave program, while the rest of the method is implemented in Perl script. The subcircuit library in Fig. 1 contains the logic of multiplexers of different sizes and the logic of LSBs of common functional modules as tabulated in Table 1. It can be seen that all library modules can be represented in canonical form with BDDs of a small number of inputs and nodes.

TABLE I. LIST OF LIBRARY MODULES

Library Module	No. of IPs	No. of OPs	No. of BDD Nodes	
2 to 1 MUX	3	1	3	
3 to 1 MUX	5	1	5	
4 to 1 MUX	6	1	7	
Incrementer	2	2	Out[0]: 1	Out[1]: 3
Decrementer	2	2	Out[0]: 1	Out[1]: 3
Adder (with carry)	5	2	Out[0]: 5	Out[1]: 8
Adder (w/o carry)	4	2	Out[0]: 3	Out[1]: 6
Subtractor (with borrow)	5	2	Out[0]: 5	Out[1]: 8
Subtractor (w/o borrow)	4	2	Out[0]: 3	Out[1]: 6
Multiplier	4	2	Out[0]: 2	Out[1]: 6

Fig. 5 depicts the block diagram of a greatest common divisor (GCD) calculator, which will be used to demonstrate the workflow of the proposed method and analyze the runtime of the candidate subcircuit enumeration algorithm (being the most time consuming stage in the proposed method). The flattened gate-level netlist of the GCD calculator is the input of the proposed method and the objective is to extract the subtractor (SUB) module from the netlist. The combination logic block identified by preliminary partitioning task (see Section II.B above) is shaded in Fig. 5. The registers parallelism identification task partitions the registers according to their enable signals (x_load, y_load and out_load) and form three register groups (X_REG, Y_REG and OUT_REG in Fig. 5), and their respective input and output signals form six signal groups (*SG1* to *SG6* in the figure).

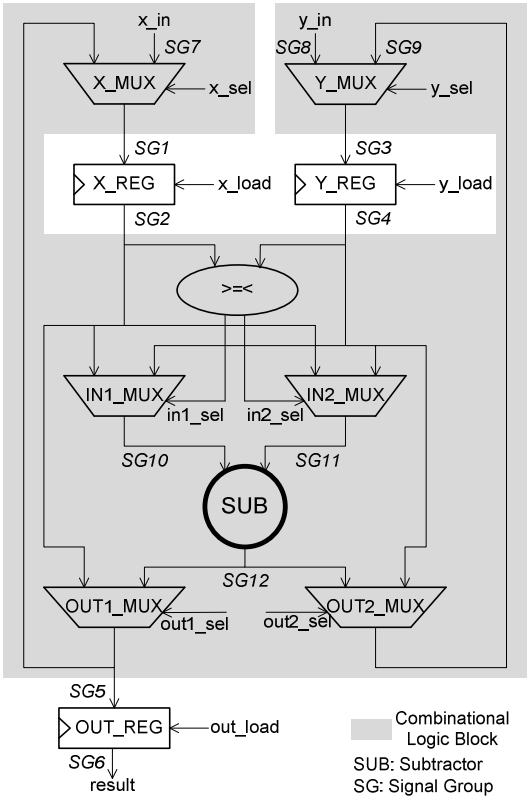


Figure 5. Block diagram of a GCD calculator, of which the SUB module is to be extracted by the proposed method.

The multiplexers in the combinational logic block are thereafter identified using the Subcircuit Recognition algorithm (see Section III above). In order to depict the effect of the size of the combinational logic block on the runtime of the candidate subcircuit enumeration algorithm, the GCD calculator is synthesized with different data widths (resulting in different combinational logic block sizes) and candidate subcircuits with 3 inputs and 1 output (to be matched with 2-to-1 MUX) are enumerated.

The runtimes of the CS enumeration algorithm for different target sizes are tabulated in Table 2. As expected, it can be seen that the runtime grows exponentially with the size of the target (in terms of number of gates/cells), which indicates the importance of the proposed analysis and simplification steps (state machine extraction & removal and preliminary partitioning) for target size reduction discussed in Section II.

Figure 6 depicts the efficacy of the various proposed techniques to speed up the basic CS enumeration algorithm (see Section III.B above). It can be seen that a total of more than 95% reduction in runtime is achieved by a combination of the three proposed techniques, the upper limit, the single-gate removal and the gate-by-gate removal.

TABLE II. RUNTIME OF CS ENUMERATION ALGORITHM FOR DIFFERENT TARGET SIZES

Data Width (No. of Bits)	Size of Target (No. of Gates/Cells)	No. of CS	Runtime (sec)
4	64	49	288
8	143	175	1475
12	211	248	7104
16	276	323	12000

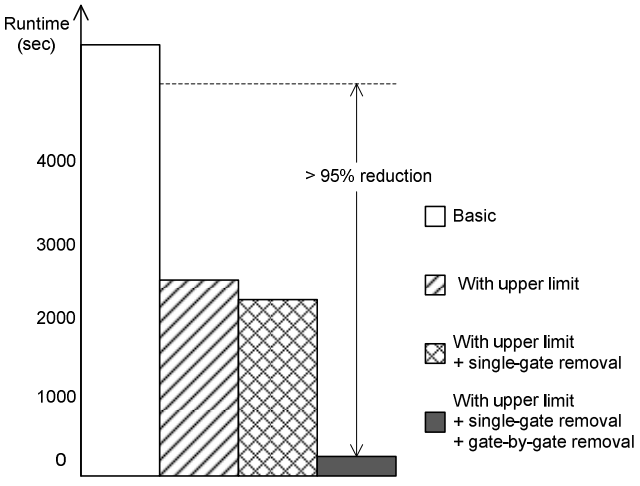


Figure 6. Improvement in runtime of CS enumeration algorithm from proposed techniques.

Cross matching (see Section III.C above) divides the 49 multi-gate candidate subcircuits in the 4-bit GCD calculator into 23 groups, with subcircuits in each group syntactically equivalent. Only two groups (out of the 23 groups), representing 16 candidate subcircuits, pass the Signature Testing (see Section III.D above), and eventually, all 2-to-1 multiplexers in the target netlist are identified by means of BDD comparison (semantic equivalence checking). In other

words, the proposed cross matching and Signature Testing steps reduce the number of candidate subcircuits to be matched to the library module by 96% from 49 to 2.

Parallelism identification & propagation task groups the multiplexers according to their select signals (x_sel, y_sel, in1_sel, in2_sel, out1_sel and out2_sel in Fig. 5) and forms six distinguished MUX groups (X_MUX, Y_MUX, IN1_MUX, IN2_MUX, OUT1_MUX and OUT2_MUX in Fig. 5). It also generates a more comprehensive signal group list comprising the previous six signal groups (*SG1* to *SG6*) and six new signal groups (*SG7* to *SG12* in Fig. 5).

Following a similar approach, all candidate subcircuits with 4 inputs and 2 outputs are enumerated and compared with various functional modules in Table 1 in the LSBs matching task. The result is a single match with the Subtractor (w/o borrow), which indicates the existence of a candidate subtractor. The candidate subtractor is expanded according to the identified signal groups *SG10*, *SG11* and *SG12* in the CFM expansion task, and finally the signals in the three groups are ordered using logic depth search. The final product is the extraction of the SUB module from the flattened gate-level netlist of the GCD calculator with its input/output signals ordered.

The proposed algorithm is tested on several benchmark circuits and the results of various tasks therein are listed in Table 3. It can be seen that most of the benchmark circuits have their functional modules identified and expanded successfully. Nonetheless, no candidate functional module can be identified in S344 because the circuit is highly optimized and the adder therein is merged with other circuits. The result suggests the direction for future improvement of the proposed algorithm.

TABLE III. LIST OF BENCHMARK CIRCUITS AND RESULTS

Benchmark Circuit	CFM Identification	Module Expansion
GCD	Pass	Pass
74283	Pass	Pass
Class D[14]	Pass	Pass
FIR Filter	Pass	Pass
S344	Fail	-

Note: 74283 – 74X series adder, Class D – Class D amplifier, S344 – ISCAS'89.

V. CONCLUSIONS

A generic and versatile method for extracting functional modules from flattened gate-level netlist has been proposed. It is advantageous over reported methods because it requires no prior knowledge about the input netlist, is fully automated, and employs a highly compact module library containing single generic model of each type of common functions with arbitrary data width. Experiment results have demonstrated the efficacy of the proposed method and proposed techniques for improving its efficiency. The future work will involve boundary reconstruction to identify functional modules in highly optimized circuits and building of a more comprehensive subcircuit library.

REFERENCES

[1] S. Novakovsky et al., "High Capacity and Automtic Functional Extraction Tool for Industrial VLSI Circuit Designs," Proc. 2002 IEEE/ACM Int. Conf. CAD, pp. 520-525, Nov. 2002.

[2] I. Parulkar et al., "Extraction of a High-Level Structural Representation from Circuit Descriptions with Applications to DFG/BIST," Proc. 31st DAC, pp. 345-356, Jun. 1994.

[3] M. Tehranipoor et al., "A Survey of Hardware Trojan Taxonomy and Detection," IEEE Design & Test of Computers, pp. 10-25, Jan./Feb. 2010.

[4] J. Kumagai, "Chip Detectives," IEEE Spectrum, pp. 43-49, Nov. 2000.

[5] T. Doom et al., "Identifying High-Level Components in Combinational Circuits," Proc. GLS on VLSI 98, pp. 313, Feb. 1998.

[6] M. Ohlrich et al.. "SubGemini: Identifying SubCircuits Using a Fast Subgraph Isomorphism Algorithm," Proc. 30th DAC, pp. 31-37, Jul. 1993.

[7] N. Rubanov, "A High-Performance Subcircuit Recognition Method Based on the Nonlinear Graph Optimization," IEEE Trans. CAD, vol. 25, no. 11, pp. 2353-2363, Nov. 2006.

[8] S.T. Eckmann et al., "Assigning Functional Meaning to Digital Circuits," ANL/DIS/TM-34, Argonne National Laboratory, Jul. 1997.

[9] G.H. Chisholm et al., "Understanding Integrated Circuits," IEEE Design & Test of Computers, vol. 16, no. 2, pp. 26-37, Apr. 1999.

[10] M.C. Hansen et al., "Unveiling the ISCAS-85 Benchmarks: A Case Study in Reverse Engineering," IEEE Design & Test of Computers, vol. 16, no. 3, pp. 72-80, Jul. 1999.

[11] Y. Shi et al., "A Highly Efficient Method for Extracting FSMs from Flattened Gate-Level Netlist," Proc. 2010 IEEE ISCAS, pp. 2610-2613, May/Jun. 2010.

[12] K.S. Brace et al., "Efficient Implementation of a BDD Package," Proc. 27th DAC, pp. 40-45, Jan. 1991.

[13] http://vlsicad.eecs.umich.edu/BK/Slots/cache/www.itu.dk/research/buddy/index.html

[14] B.-H. Gwee et al., "A Micropower Low-Distortion Digital Class-D Amplifier Based on an Algorithmic Pulsewidth Modulator," IEEE Trans. CAS, vol. 52, no. 10, pp. 2007-2022, Oct. 2005.